UNIVERSIDAD DEL QUINDÍO — ANÁLISIS DE ALGORITMOS

# GUÍA MAESTRA DE SUSTENTACIÓN Y DEFENSA TÉCNICA

*Herramienta personal de estudio: Código, Algoritmos, Matemáticas y Preguntas Trampa del Docente*

## Módulo 1: El Pitch Inicial (Tu Apertura de 2 Minutos)

Si el docente te dice: 'Explícame qué hiciste en el taller y qué demostraste', di exactamente esto con seguridad:

«Profesor, el objetivo de este taller fue procesar un corpus no estructurado de 20 artículos científicos en PDF sobre Inteligencia Artificial Generativa en educación, extrayendo el abstract de cada uno para construir un ranking bibliométrico de términos más frecuentes.

El núcleo del reto algorítmico consistió en ordenar las frecuencias de 1.158 términos únicos evitando por completo algoritmos basados en comparaciones (como MergeSort o QuickSort), los cuales tienen un piso teórico infranqueable de Ω(n log n). En su lugar, implementamos tres algoritmos de tiempo lineal: Counting Sort, Radix Sort y Bucket Sort.

Comprobamos matemáticamente y de forma empírica que, como las frecuencias son enteros positivos en un rango acotado (aquí k=66, donde k << n), los tres algoritmos logran ordenar en tiempo O(n + k) u O(n), tardando menos de 1.1 milisegundos en la máquina real y coincidiendo el 100% en el Top 15 de términos arrojado.»

## Módulo 2: Fundamentación Teórica que el Docente Preguntará

📌 **PREGUNTA 1: ¿Por qué los algoritmos de comparación como QuickSort o MergeSort no pueden bajar de Ω(n log n)?**

Respuesta Clave: Se demuestra mediante el Modelo de Árbol de Decisión. Cualquier algoritmo que solo compara pares (¿A[i] <= A[j]?) debe ser capaz de distinguir entre las n! permutaciones posibles de los n elementos. Por lo tanto, el árbol de decisión binario debe tener al menos L >= n! hojas. Como la altura h del árbol representa el número de comparaciones en el peor caso y 2^h >= L >= n!, tomando logaritmo tenemos h >= log2(n!). Por la aproximación de Stirling (ln(n!) ≈ n ln n - n), se concluye rigurosamente que h = Ω(n log n).

📌 **PREGUNTA 2: ¿Cómo 'burlan' o rompen Counting, Radix y Bucket Sort esa barrera?**

Respuesta Clave: No la violan porque NO pertenecen al modelo de comparación. Esos algoritmos aprovechan información extra: las claves son números enteros positivos acotados en [1..k]. En vez de

comparar si una frecuencia es mayor que otra, usan la frecuencia directamente como ÍNDICE en memoria (direccionamiento directo). El acceso a una posición de arreglo count[freq] o bucket[freq] toma tiempo constante O(1), permitiendo construir el arreglo ordenado en O(n + k) pasos.

📌 **PREGUNTA 3: ¿Qué es la 'Estabilidad' y por qué fue indispensable en tu implementación?**
Respuesta Clave: Un algoritmo es estable si conserva el orden relativo original de elementos que tienen la misma clave (frecuencia). En nuestro corpus, muchas palabras tienen exactamente la misma frecuencia (por ejemplo, 'future' y 'systems' tienen 15 ocurrencias). Si no mantuviéramos la estabilidad, el Top 15 cambiaría de orden arbitrariamente entre algoritmos. Hicimos que Counting Sort y Radix Sort recorrieran de derecha a izquierda (n-1 hasta 0) para que la palabra que apareció primero en el texto quede primero en la salida final. Por eso el Top 15 de los 3 coincidió al 100%.

## Módulo 3: Explicación Detallada del Código (Línea por Línea)

### 3.1. Extracción y Preprocesamiento de PDFs

• Librería pypdf: Se utilizó para abrir los 20 documentos de forma limpia. Se leen las 2 primeras páginas porque allí reside universalmente el abstract.
• Descomposición Unicode (NFKD): Algunos PDFs científicos contienen 'ligaduras tipográficas' (por ejemplo, la combinación 'fi' o 'fl' como un solo glifo UTF-8 como fi). Con unicodedata.normalize('NFKD', text) descomponemos esos caracteres raros en texto estándar 'f' e 'i', evitando errores de lectura.
• Expresión Regular para Abstract: Se usó re.search(r'(?:ABSTRACT|Abstract)...(?:Keywords|Introduction)', text, re.DOTALL) para recortar con exactitud el resumen.
• Limpieza y Tokenización: re.findall(r'\b[a-z]{3,}\b', text.lower()) extrae solo palabras alfabéticas de 3 o más letras.
• Stopwords: Se filtraron pronombres y conectores ('the', 'and', 'in') y términos recurrentes de redacción académica ('paper', 'study', 'results') para que el ranking capture el dominio real de IA y educación.

### 3.2. Counting Sort Descendente — Función counting_sort_descending()

¿Cómo funciona paso a paso?
1. Hallar k: k = max(item[1] for item in arr). En nuestro corpus k = 66.
2. Arreglo de Conteo: count = [0] * (k + 1). O(k) en memoria y tiempo.
3. Llenar Conteo: Por cada tupla (palabra, freq), hacemos count[freq] += 1. Esto toma O(n).
4. Acumulación Inversa (Descendente): En lugar de acumular de izquierda a derecha (ascendente), acumulamos de derecha a izquierda:
  for i in range(max_freq - 1, -1, -1): count[i] += count[i + 1]
  Ahora count[i] contiene exactamente cuántas palabras tienen una frecuencia >= i. Esto indica en qué índice colocarlo para que los mayores vayan primero.
5. Construcción Estable: Recorremos el arreglo original de atrás hacia adelante (n-1 hasta 0), calculamos pos = count[freq] - 1, insertamos en output[pos] = item, y decrementamos count[freq] -= 1.
• Complejidad Total: O(n + k) en tiempo y O(n + k) en espacio auxiliar.

### 3.3. Radix Sort LSD Descendente — Función radix_sort_descending()

¿Cómo funciona paso a paso?
1. LSD (Least Significant Digit): Ordena número por número empezando por el dígito menos significativo (unidades) y avanzando hacia el más significativo (decenas, centenas).
2. Variable exp: Comienza en exp = 1 (unidades). En la siguiente iteración exp = 10 (decenas). El ciclo termina cuando max_freq // exp == 0. Como k=66, solo realiza 2 pasadas (d=2 dígitos).
3. Subrutina de conteo por dígito: En cada pasada, extrae el dígito actual con digit = (freq // exp) % 10. Aplica un conteo de base b=10 (arreglo de tamaño 10) acumulando de forma inversa y colocando elementos de derecha a izquierda para mantener la estabilidad.

• Complejidad Total: O(d · (n + b)). Como d=2 y b=10 son constantes pequeñas fijas, su comportamiento es efectivamente lineal O(n).

### 3.4. Bucket Sort — Función bucket_sort_descending()

¿Cómo funciona paso a paso?

1. Creación de Cubetas: buckets = [[] for _ in range(max_freq + 1)]. Se crea una lista/cubeta para cada frecuencia posible entre 0 y k (67 cubetas).
2. Distribución directa: for item in arr: buckets[item[1]].append(item). Como la clave es un número entero exacto, cada palabra entra directamente a la cubeta de su frecuencia en tiempo O(1). No se requiere ningún Insertion Sort adentro porque todos los elementos dentro de la cubeta tienen la misma frecuencia.
3. Concatenación Inversa: Recorremos las cubetas desde k=66 hacia 0 extrayendo los elementos. Como se agregan primero los de mayor frecuencia, el resultado queda perfectamente ordenado de mayor a menor.

• Complejidad Total: O(n + k) tiempo y O(n + k) memoria.

## Módulo 4: Análisis de Resultados Empíricos (Preguntas de Gráficas)

| Algoritmo | Tiempo Promedio (ms) | Complejidad Teórica | ¿Por qué dio ese tiempo? |
|---|---|---|---|
| **Bucket Sort** | 0.2845 ms | O(n + k) | Más rápido. Usa listas de Python cuyos append() están implementados en C a bajo nivel sin aritmética. |
| **Counting Sort** | 0.4648 ms | O(n + k) | Excelente tiempo. Hace exactamente 2 recorridos lineales y cálculo de posiciones pos = count[freq] - 1. |
| **Radix Sort** | 1.0681 ms | O(d · (n + b)) | Más lento de los tres. Debe ejecutar la subrutina de conteo 2 veces completas (d=2), duplicando asignaciones. |

## Módulo 5: Banco de Preguntas Trampa del Docente y Respuestas Exactas

📌 **CASCARITA 1: ¿Qué pasaría con Counting Sort si tuviéramos solo 50 palabras pero una de ellas tiene frecuencia de 10.000.000?**

Respuesta Imbatible: 'Profesor, Counting Sort dejaría de ser eficiente y colapsaría. Su complejidad es O(n + k). Si n = 50 pero k = 10.000.000, tendríamos que crear un arreglo auxiliar de 10 millones de posiciones para ordenar solo 50 elementos, desperdiciando memoria masiva y tardando 10 millones de iteraciones en acumular. Counting Sort solo es competitivo cuando k <= n o k = O(n), lo cual se cumplió perfectamente en nuestro taller donde n=1.158 y k=66'.

📌 **CASCARITA 2: ¿En Bucket Sort clásico del libro de Cormen no se usa Insertion Sort dentro de cada cubeta? ¿Por qué aquí no?**

Respuesta Imbatible: 'Muy buena observación, profesor. En el Cormen (Cap. 8.4) Bucket Sort asume números reales continuos en [0, 1), por lo que varias claves distintas caen en el mismo intervalo y se

necesita Insertion Sort interno. Sin embargo, en nuestro problema bibliométrico las claves no son continuas sino enteras discretas acotadas (1, 2, ..., 66). Por lo tanto, asignamos exactamente una cubeta a cada valor entero posible (Principio del Palomar o Pigeonhole). Como todos los elementos dentro de la cubeta 'x' tienen idéntica frecuencia x, ¡el orden interno ya está garantizado y no requiere comparaciones!'.

**📌 CASCARITA 3: ¿Por qué en Radix Sort procesas de Menor a Mayor Significativo (LSD) y no al revés (MSD)?**

Respuesta Imbatible: 'Porque LSD permite que cada pasada preserve el orden relativo de los dígitos anteriores gracias a la estabilidad. Si ordenáramos por el dígito más significativo primero (MSD) de forma plana, la segunda pasada por las unidades destruiría el orden de las decenas a menos que usemos recursión dividiendo en sub-arreglos (lo que añade sobrecosto de pila)'.

**📌 CASCARITA 4: ¿Por qué para medir los tiempos ejecutas 50 iteraciones y no solo una?**

Respuesta Imbatible: 'Para evitar el ruido experimental del sistema operativo. Una sola ejecución puede verse afectada por interrupciones de la CPU, cambio de contexto de procesos o el recolector de basura de Python. Promediar 50 iteraciones tras un calentamiento de memoria nos da una medida estadísticamente confiable del tiempo real de CPU'.

**📌 CASCARITA 5: ¿Qué relevancia bibliométrica tuvieron los resultados obtenidos?**

Respuesta Imbatible: 'El Top 3 de términos fue 'education' (66), 'aied' (55) y 'learning' (45). Esto demuestra de forma cuantitativa que el corpus de 20 artículos está fuertemente focalizado en la Inteligencia Artificial Aplicada a la Educación (AIED). Además, la aparición de 'assessment' (#5) y 'human' (#15) ratifica las tendencias actuales hacia la evaluación mediante IA y los marcos centrados en el ser humano (Human-Centred AI)'.